

Trabajo Práctico Integrador Final

Alumna

Benitez Nayla

Tecnicatura Universitaria en Programación - Universidad Tecnológica Nacional.

Organización Empresarial

Docente Titular

Carolina Bruno

Docente Tutor

Roberto Regner

Junio 2026

Tabla de contenido

Introducción	3
Objetivos (consigna y sistema)	4
Desarrollo	5
Conclusión	18
Referencias	19

Trabajo Práctico Integrador Final – Organización Empresarial

Introducción

Este es un trabajo práctico integrador perteneciente a la materia Organización Empresarial (OE), realizado por Benitez Nayla.

El presente Trabajo Práctico Integrador tiene como objetivo analizar y automatizar el proceso de solicitud de vacaciones dentro de una organización mediante la utilización de un chatbot.

Actualmente, la gestión de vacaciones suele realizarse de forma manual, requiriendo la intervención del área de Recursos Humanos para verificar la disponibilidad de días y aprobar o rechazar solicitudes. Esta situación genera demoras y aumenta la carga administrativa.

Como solución se propone un chatbot capaz de consultar una base de datos simulada, validar las reglas de negocio y brindar una respuesta inmediata al empleado. El diseño se modela mediante BPMN 2.0 para garantizar la coherencia entre el proceso de negocio y la lógica implementada.

Objetivo de la consigna:

Deberán identificar un proceso administrativo crítico dentro de una organización (ej. gestión de vacaciones, soporte técnico nivel 1, alta de proveedores) y automatizarlo mediante un chatbot, asegurando que la lógica del bot responda a un modelo de procesos estandarizado.

Realizarlo como Simulación de proceso

Objetivo del sistema generado:

Desarrollar un chatbot capaz de automatizar el proceso de solicitud de vacaciones.

El sistema permitirá:

- Consultar la información del empleado.
- Verificar la cantidad de días disponibles.
- Aprobar o rechazar solicitudes según las reglas definidas.
- Informar el resultado de manera inmediata.

Desarrollo:**1. Modelado de Negocio (El "Qué")**

- Identificación del Proceso:

Descripción del problema

Actualmente los empleados solicitan vacaciones mediante correo electrónico o formularios internos. El área de Recursos Humanos debe verificar manualmente los días disponibles y comunicar la aprobación o rechazo de la solicitud.

Este procedimiento genera demoras, aumenta la carga administrativa y puede provocar errores en la gestión de la información.

Objetivo

Automatizar el proceso de solicitud de vacaciones mediante un chatbot que consulte una base de datos simulada, valide reglas de negocio y responda automáticamente al usuario.

REGLAS DE NEGOCIO**RN1**

El legajo debe existir.

RN2

La cantidad de días solicitados debe ser mayor que cero.

RN3

La cantidad de días solicitados no puede superar los días disponibles.

RN4

Si todas las validaciones son correctas, la solicitud será aprobada.

RN5

Si alguna validación falla, la solicitud será rechazada.

- Modelado de negocio (BPMN 2.0):

Proceso Actual (AS-IS)

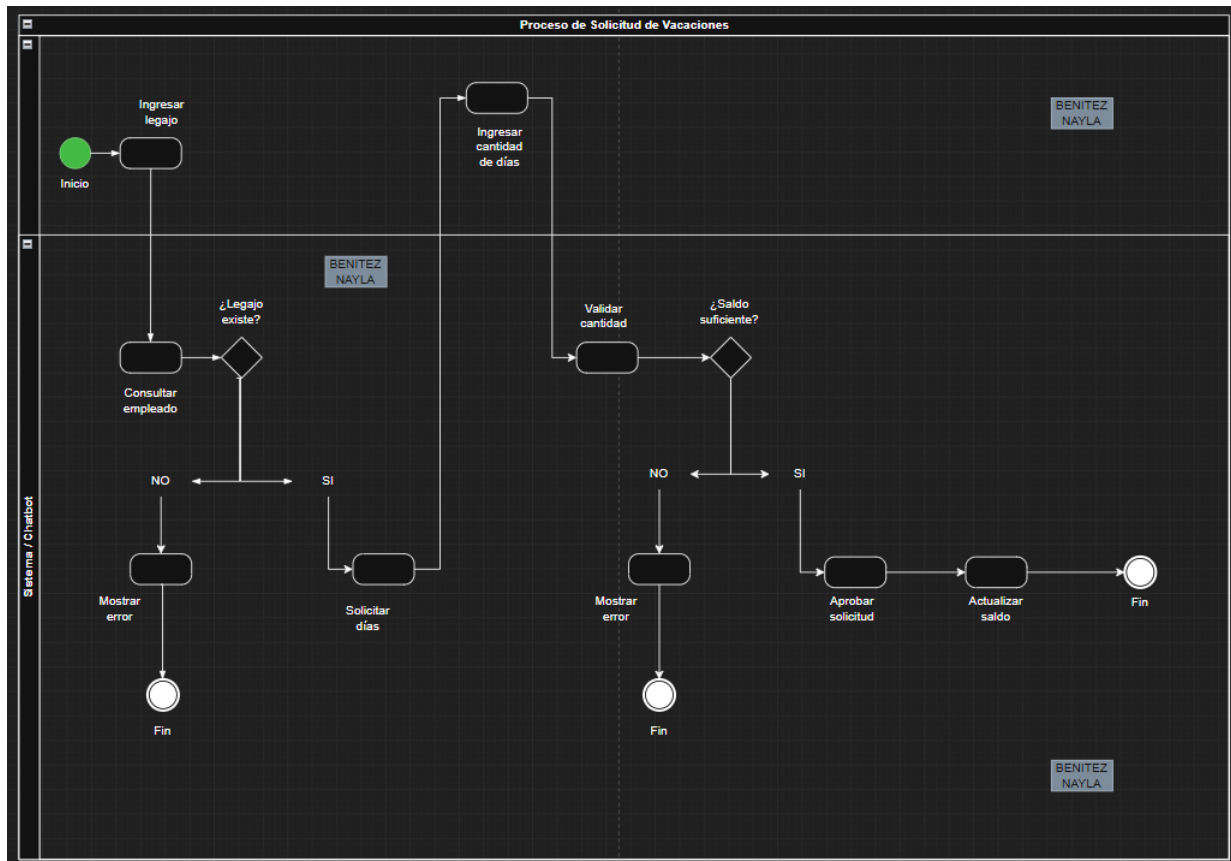
1. El empleado envía una solicitud de vacaciones.
2. Recursos Humanos recibe la solicitud.
3. Recursos Humanos consulta los días disponibles.
4. Recursos Humanos analiza la solicitud.
5. Recursos Humanos aprueba o rechaza el pedido.
6. El empleado recibe una respuesta.

Problemas detectados

- Demoras en la respuesta.
- Trabajo manual repetitivo.
- Posibles errores humanos.
- Dependencia de la disponibilidad del personal de RRHH.

Proceso Mejorado (TO-BE)

1. El empleado inicia una conversación con el chatbot.
2. El chatbot solicita el número de legajo.
3. El sistema consulta la base de datos.
4. El chatbot solicita la cantidad de días deseados.
5. El sistema valida la información.
6. El chatbot aprueba o rechaza la solicitud automáticamente.

Diagrama BPMN:

LINK Diagrama:

<https://drive.google.com/file/d/1gZ8sPiWZrby6NSnU1YUL9wjfiNxEf2Ak/view?usp=sharing>

2. Arquitectura y Programación (El "Cómo") (esto es integración con las otras materias)

- Selección del Stack:

Plataforma y Lenguaje

Se propone Python como lenguaje de programación por su simplicidad y facilidad para implementar lógica conversacional.

```
chatbot_vacaciones.py > ...
1  import csv
2
3
4  def cargar_empleados():
5
6      empleados = {}
7
8      with open(
9          "empleados.csv",
10         mode="r",
11         encoding="utf-8"
12     ) as archivo:
13
14         lector = csv.DictReader(
15             archivo,
16             delimiter=";"
17         )
18
19         for fila in lector:
20
21             legajo = int(fila["Legajo"])
22
23             empleados[legajo] = {
24                 "nombre": fila["Nombre"],
25                 "dias_disponibles": int(
26                     fila["Dias_Disponibles"]
27                 ),
28                 "email": fila["Email"]
29             }
30
31     return empleados
32
33
34     empleados = cargar_empleados()
35
36     estado = "ESPERANDO_LEGAJOS"
37
38     print("=" * 40)
39     print("CHATBOT DE SOLICITUD DE VACACIONES")
40     print("=" * 40)
41
42     while True:
43
44         if estado == "ESPERANDO_LEGAJOS":
45
46             try:
47                 legajo = int(input("Ingrese su legajo: "))
48             except ValueError:
```

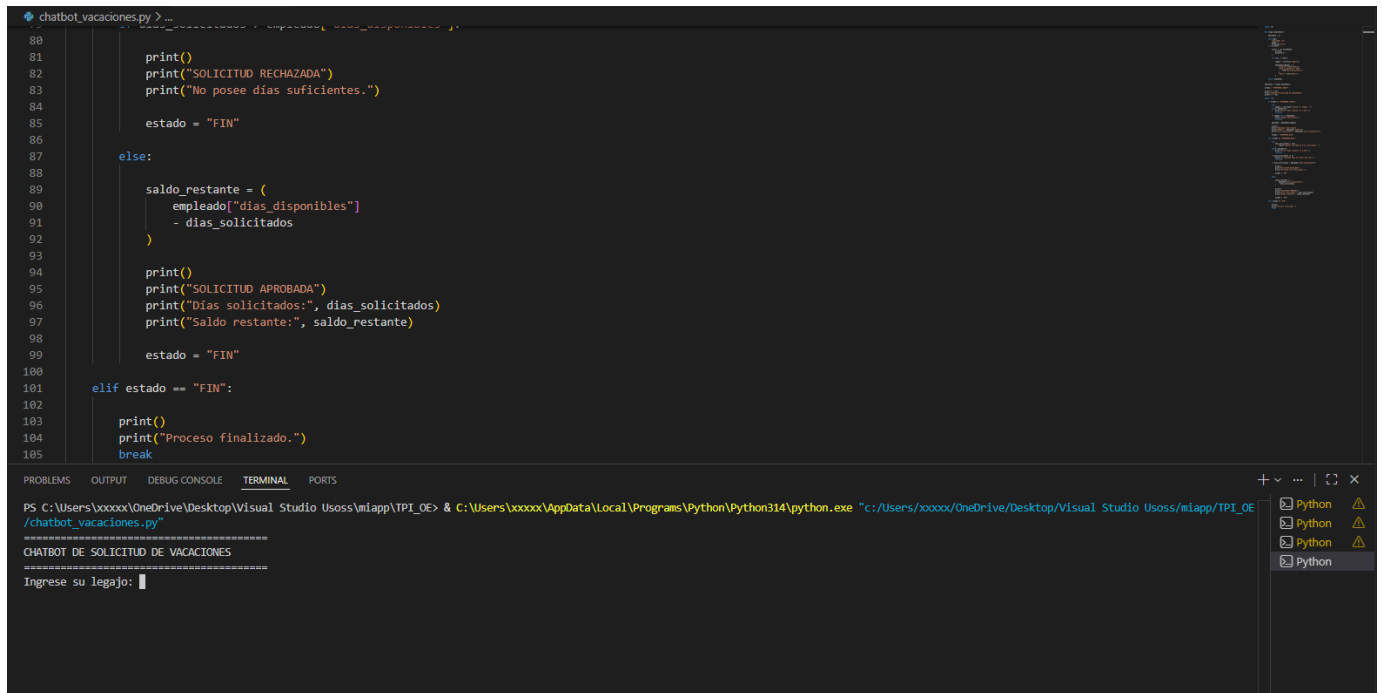


```
chatbot_vacaciones.py > ...
48     except ValueError:
49         print("Error: debe ingresar un número.")
50         continue
51
52     if legajo not in empleados:
53         print("Legajo inexistente.")
54         continue
55
56     empleado = empleados[legajo]
57
58     print()
59     print("Empleado encontrado")
60     print("Nombre:", empleado["nombre"])
61     print("Días disponibles:", empleado["dias_disponibles"])
62
63     estado = "ESPERANDO_DIAS"
64
65     elif estado == "ESPERANDO_DIAS":
66
67         try:
68             dias_solicitados = int(
69                 input("Ingrese cantidad de días solicitados: ")
70             )
71         except ValueError:
72             print("Error: debe ingresar un número.")
73             continue
74
75         if dias_solicitados <= 0:
76             print("La cantidad debe ser mayor que cero.")
77             continue
78
79         if dias_solicitados > empleado["dias_disponibles"]:
80
81             print()
82             print("SOLICITUD RECHAZADA")
83             print("No posee días suficientes.")
84
85             estado = "FIN"
86
87         else:
88
89             saldo_restante = (
90                 empleado["dias_disponibles"]
91                 - dias_solicitados
92             )
93
94             print()
95             print("SOLICITUD APROBADA")
```

```
95         print("SOLICITUD APROBADA")
96         print("Días solicitados:", dias_solicitados)
97         print("Saldo restante:", saldo_restante)
98
99         estado = "FIN"
100
101     elif estado == "FIN":
102
103         print()
104         print("Proceso finalizado.")
105         break
106
```

El chatbot podría desplegarse en Telegram, WhatsApp Business o una aplicación web.

Para este trabajo se implementa una simulación por consola.



```
chatbot_vacaciones.py > ...
80
81     print()
82     print("SOLICITUD RECHAZADA")
83     print("No posee días suficientes.")
84
85     estado = "FIN"
86
87 else:
88
89     saldo_restante = (
90         empleado["días disponibles"]
91         - días_solicitados
92     )
93
94     print()
95     print("SOLICITUD APROBADA")
96     print("Días solicitados:", días_solicitados)
97     print("Saldo restante:", saldo_restante)
98
99     estado = "FIN"
100
101 elif estado == "FIN":
102
103     print()
104     print("Proceso finalizado.")
105     break

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\xxxxx\OneDrive\Desktop\Visual Studio Usoss\miapp\TPI_OE> & C:\Users\xxxxx\AppData\Local\Programs\Python\Python314\python.exe "c:/Users/xxxxx/OneDrive/Desktop/Visual Studio Usoss/miapp/TPI_OE/chatbot_vacaciones.py"
CHATBOT DE SOLICITUD DE VACACIONES
=====
Ingrese su legajo: 
```

Arquitectura de la solución

La solución se compone de tres elementos principales:

- Chatbot desarrollado en Python.
- Archivo CSV utilizado como fuente de datos.
- Modelo BPMN que representa el proceso de negocio.

El chatbot consulta la información almacenada en el archivo CSV, aplica las reglas de negocio definidas y genera una respuesta automática para el usuario.

- Integración de Datos:

Persistencia de datos

Los datos se almacenan en un archivo CSV denominado empleados.csv.

La planilla fue creada inicialmente mediante Microsoft Excel y exportada a formato CSV para su

utilización desde Python. La estructura contiene el número de legajo, nombre del empleado y cantidad de días disponibles para solicitar vacaciones.

El chatbot consulta dicho archivo para validar la existencia del empleado y verificar la disponibilidad de días de vacaciones, de este modo la interacción con la planilla no es simulada.

Legajo	Nombre	Días_Disponibles	Email
1001	Juan Perez	15	Juan@empresa.com
1002	Ana Lopez	5	Ana@empresa.com
1003	Pedro Gome	20	Pedro@empresa.com
1004	Maria Diaz	10	Maria@empresa.com
1005	Carlos Ruiz	8	Carlos@empresa.com
1006	Sofia Marti	12	Sofia@empresa.com
1007	Lucas Fernan	18	Lucas@empresa.com
1008	Valentina To	7	Valentina@empresa.com
1009	Mateo Alvar	25	Mateo@empresa.com
1010	Camila Rome	14	Camila@empresa.com

Diccionario de datos:

Variable	Tipo	Descripción
legajo	int	Identificador único del empleado
nombre	str	Nombre del empleado
días_disponibles	int	Cantidad de días restantes
días_solicitados	int	Días solicitados por el usuario
estado	str	Estado actual de la conversación

GESTIÓN DE ESTADOS

La conversación se modela mediante una Máquina de Estados.

INICIO

↓

ESPERANDO_LEGajo

↓

VALIDANDO_LEGajo

↓

ESPERANDO_DIAS

↓

VALIDANDO_DIAS

↓

APROBADO

o

RECHAZADO

↓

FIN

CAMINO FELIZ

Legajo válido

↓

Cantidad válida

↓

Saldo suficiente

↓

Solicitud aprobada

↓

Cálculo del saldo restante

↓

Fin

CAMINO INFELIZ

Caso 1

Legajo inexistente

Respuesta:

Legajo no encontrado.

Caso 2

Cantidad negativa

Respuesta:

La cantidad debe ser mayor que cero.

Caso 3

Texto en lugar de número

Respuesta:

Debe ingresar un valor numérico.

Caso 4

Saldo insuficiente

Respuesta:

Solicitud rechazada por falta de días disponibles.

Herramientas Utilizadas

Durante el desarrollo del proyecto se utilizaron las siguientes herramientas:

- Python 3.14 para la implementación del chatbot. 
- Visual Studio Code como entorno de desarrollo. 
- Draw.io para el modelado BPMN. 
- GitHub para el control de versiones y almacenamiento del proyecto. 

- Microsoft Excel para la creación inicial de los datos.



- Archivo CSV como mecanismo de persistencia.



Casos de Prueba

Se realizaron pruebas funcionales sobre el chatbot contemplando escenarios válidos e inválidos.

Entre los casos evaluados se encuentran:

- Solicitud aprobada con saldo suficiente.
- Solicitud rechazada por falta de días disponibles.
- Ingreso de legajo inexistente.
- Ingreso de valores no numéricos.
- Solicitud de días negativos o iguales a cero.

Los resultados obtenidos fueron satisfactorios, verificando el correcto funcionamiento de las reglas de negocio definidas para el proceso.

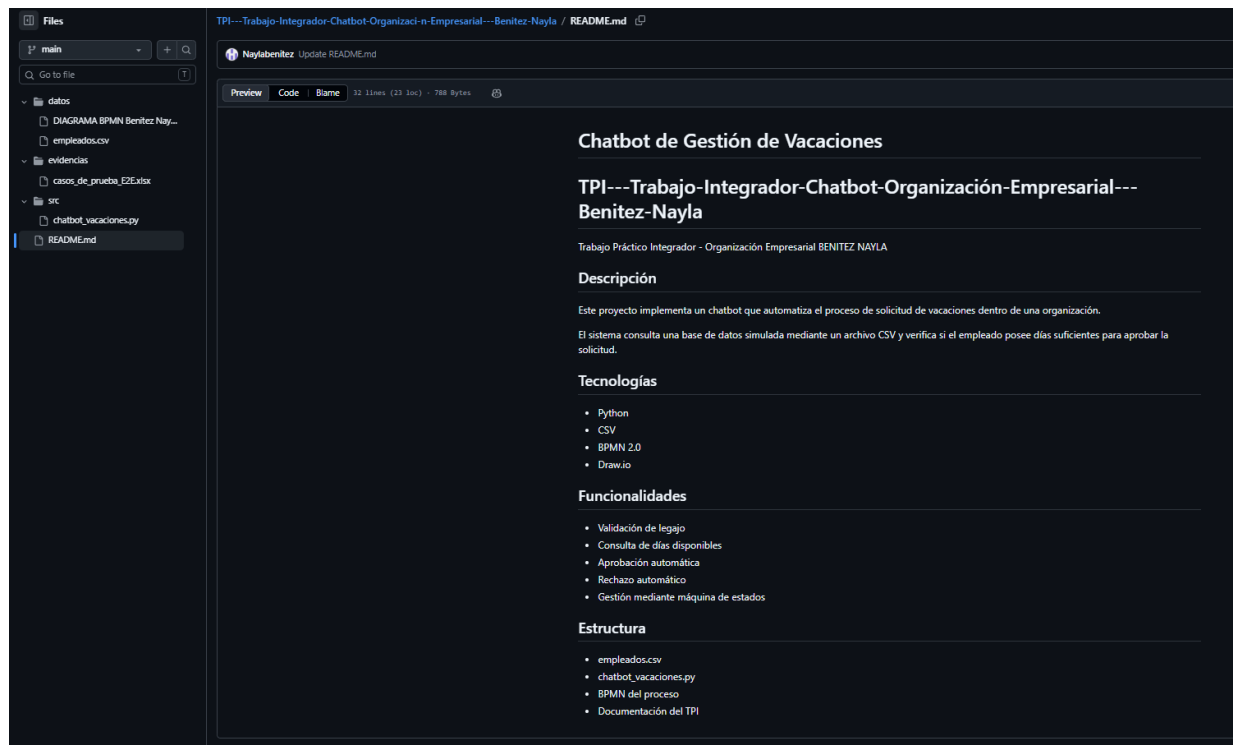
Caso	Entrada	Resultado Esperado	Resultado Obtenido	Estado	Evidencias
CP-01	Legajo 1001, 50 días	Rechazo por saldo insuficiente	Solicitud rechazada	☒ OK	CHATBOT DE SOLICITUD DE VACACIONES ===== Ingrese su legajo: 1001 Empleado encontrado Nombre: Juan Perez Días disponibles: 15 Ingrese cantidad de días solicitados: 50 SOLICITUD RECHAZADA No posee días suficientes. Proceso finalizado
CP-02	Legajo 1001, 5 días	Aprobación	Solicitud aprobada	☒ OK	CHATBOT DE SOLICITUD DE VACACIONES ===== Ingrese su legajo: 1001 Empleado encontrado Nombre: Juan Perez Días disponibles: 15 Ingrese cantidad de días solicitados: 5 SOLICITUD APROBADA Días solicitados: 5 Saldo restante: 10 Proceso finalizado.
CP-03	Legajo 1002, 10 días	Rechazo por saldo insuficiente	Solicitud rechazada	☒ OK	CHATBOT DE SOLICITUD DE VACACIONES ===== Ingrese su legajo: 1002 Empleado encontrado Nombre: Ana Lopez Días disponibles: 5 Ingrese cantidad de días solicitados: 10 SOLICITUD RECHAZADA No posee días suficientes. Proceso finalizado.

CP-04	Legajo 9999	Error de legajo inexistente	Legajo inexistente	☒ OK	CHATBOT DE SOLICITUD DE VACACIONES =====
					Ingrese su legajo: 9999 Legajo inexistente.
CP-05	Legajo 1000	Error de legajo inexistente	Legajo inexistente	☒ OK	CHATBOT DE SOLICITUD DE VACACIONES =====
					Ingrese su legajo: 1000 Legajo inexistente.
CP-06	Legajo "hola"	Error de formato	Debe ingresar un número	☒ OK	CHATBOT DE SOLICITUD DE VACACIONES =====
					Ingrese su legajo: hola Error: debe ingresar un número.
CP-07	Legajo 1003, -5 días	Error de validación	Cantidad mayor que cero	☒ OK	CHATBOT DE SOLICITUD DE VACACIONES =====
					Ingrese su legajo: 1003 Empleado encontrado Nombre: Pedro Gomez Días disponibles: 20 Ingrese cantidad de días solicitados: -5 La cantidad debe ser mayor que cero.
CP-08	Legajo 1003, 0 días	Error de validación	Cantidad mayor que cero	☒ OK	CHATBOT DE SOLICITUD DE VACACIONES =====
					Ingrese su legajo: 1003 Empleado encontrado Nombre: Pedro Gomez Días disponibles: 20 Ingrese cantidad de días solicitados: 0 La cantidad debe ser mayor que cero.

CP-09	Legajo 1003, "cero"	Error de formato	Debe ingresar un número	☒ OK	CHATBOT DE SOLICITUD DE VACACIONES =====
					Ingrese su legajo: 1003 Empleado encontrado Nombre: Pedro Gomez Días disponibles: 20 Ingrese cantidad de días solicitados: cero Error: debe ingresar un número.
CP-10	Legajo 1003, 20 días	Aprobación con saldo 0	Solicitud aprobada	☒ OK	CHATBOT DE SOLICITUD DE VACACIONES =====
					Ingrese su legajo: 1003 Empleado encontrado Nombre: Pedro Gomez Días disponibles: 20 Ingrese cantidad de días solicitados: 20 SOLICITUD APROBADA Días solicitados: 20 Saldo restante: 0

Almacenamiento de archivos

Con motivo de almacenar en un lugar específico el diagrama de flujo BPMN, la planilla csv., el código fuente del chatbot creado en Python y adicionalmente la evidencia en Excel de los casos de pruebas realizados sobre el chatbot, se optó por crear un repositorio en la herramienta GitHub:



LINK REPOSITORIO GITHUB:

<https://github.com/Naylabenitez/TPI---Trabajo-Integrador-Chatbot-Organizaci-n-Empresarial---Benitez-Nayla.git>

Conclusión:

El desarrollo del chatbot permitió automatizar el proceso de solicitud de vacaciones mediante la aplicación de reglas de negocio definidas previamente.

La solución propuesta reduce la intervención manual, disminuye la posibilidad de errores y proporciona respuestas inmediatas a los usuarios.

Asimismo, el modelado BPMN permitió representar gráficamente el flujo del proceso y validar la coherencia entre la lógica de negocio y la implementación realizada en Python.

Referencias

Universidad Tecnológica Nacional. Material de estudio de Organización Empresarial y Programación I.

BPMN 2.0 Specification.

Draw.io Documentation

<https://www.drawio.com/>

GitHub Repository del Proyecto

<https://github.com/Naylabenitez/TPI---Trabajo-Integrador-Chatbot-Organizaci-n-Empresarial--Benitez-Nayla.git>

Python Software Foundation

Material de cátedra de Organización Empresarial.

Capturas obtenidas mediante Excel, GitHub, VisualStudioCode, Draw.io

Python Documentation: <https://docs.python.org/3/>

Git Documentation: <https://git-scm.com/doc>

GitHub Documentation: <https://docs.github.com/>

Tecnicatura Universitaria en Programación. Organización Empresarial. Trabajo Práctico Integrador Final. (2026). (1° año.). Universidad Tecnológica Nacional.

